Enums

public enum EventCategory
CONCERT, CONFERENCE, SPORTS

public enum BookingStatus
ACTIVE, CANCELLED

public enum MembershipType
REGULAR, PREMIUM

Copying objects will result in a logic error. So directly return to references so that we can modify them later.

Booking
Attributes -bookingID: String -customer: Customer -event: Event -ticketQuantity: int -originalTotalPrice: double -finalPaidAmount: double -bookingStatus: BookingStatus
Constructors Copy Constructor Parameterized Constructor
Methods +Getters() return directly to objects (no need for copying) +reserveTicket(int amount): void //reserves tickets +setBookingStatus(BookingStatus bookingStatus): void

Event
Attributes -eventID: int -eventName: String -eventCategory: EventCategory (Enum) -capacity: int -ticketPrice: double -reservedTickets: int
Constructors Copy Constructor Parameterized Constructor
Methods +Getters() return directly to objects (no need for copying) +reserveTicket(int amount): void //reserves tickets +releaseReservedTickets(int: amount): void //releases the reserved tickets +equals(Object object): boolean //overridden equals method (check eventIDs)

Customer
Attributes -customerID: int -customerName: String -balance: double
Constructors Copy Constructor Parameterized Constructor

Methods
+Getters() return directly to objects (no need for copying)
+setBalance(double balance): void //a setter for balance, validate balance argument
+canPay(double amount): boolean
+buyTicket(double ticketPrice) //if canPay() then buy it
+refund(double amount) //refunds the given amount
+equals(Object object): boolean //overridden equals method (check customerIDs)

«extends»

PremiumCustomer
Constants +premiumDiscountPercentage: int = 5;
Constructors Copy & Parameterized Constructors
@Override +buyTicket(double ticketPrice) //apply the premium discount

FileIO
Will have static methods that will read the files and return proper array lists. (It is already finished.)

OperationAttempt
Attributes -booking: Booking -isSuccessful: boolean -failReason: String
Constructors Parameterized Constructor
Methods + just getters (no copying again!)

Query
Attributes -events: ArrayList<Event> -customers: ArrayList<Customer> -activeBookings: ArrayList<Booking>
Constructors Parameterized Constructor
Instance Methods +printMostPopularEvent(): void //make sure to print valid according to homework desc. +printHighestRevenueEvent(): void +printTopCustomer(): void +printTicketsByCategory(): void **Static Methods (Utils)** +deepCopyEvents(ArrayList<Event> events): ArrayList<Event> +deepCopyCustomers(ArrayList<Customer> customers): ArrayList<Customer> +deepCopyActiveBookings(ArrayList<Booking> bookings): ArrayList<Booking> -// this only copies the active booking (cancelled ones are excluded)

BookingSystem
Attributes -events: ArrayList<Event> -customers: ArrayList<Customer> -bookings: ArrayList<Booking> -restrictedCategory: EventCategory
Constructors Parameterized Constructor A condition for group specific rules like: public BookingSystem(..., EventCategory restrictedCategory)

Instance Methods
+printSystemInfo() //prints the info, check homework desc.
+executeCommonOperations() // (create last) check homework desc.
+executeGroupOperations()
+getCustomerFromID(int customerID): Customer
+getEventFromUD(int eventID): Event
+createBooking(Customer customer, Event event, int ticketQuantity): OperationAttempt
-//If the conditions are satisfied; creates an ACTIVE booking, adds it to the bookings
-//list. Always returns to a OperationAttempt (whether it is successful or not)
+cancelBooking(Booking booking): OperationAttempt
Static Methods (Util)
+createBookingID(Customer customer, Event event): String //creates a unique id

TicketBookingApp
Will contain the main and execute the operations